



(3) Programación asistida con IA en RStudio

José Miguel Horcas Aguilera
Universidad de Málaga

**Plasencia 2026 – Curso de Análisis y Visualización de Datos: Estadística
Práctica con R e Inteligencia Artificial**

Introducción

Instalación paso a paso
Casos de uso prácticos en R
Consideraciones
Ejercicios

¿Qué es GitHub Copilot?
¿Cómo funciona GitHub Copilot?
¿Por qué usar Copilot con R?

Introducción

Introducción

¿Qué es GitHub Copilot?



GitHub Copilot

- Es un asistente de programación basado en IA.
- Fue desarrollado por GitHub en colaboración con OpenAI.
- Proporciona sugerencias de código mientras programamos.
- Se integra con editores como VSCode, Neovim, o RStudio.

Introducción

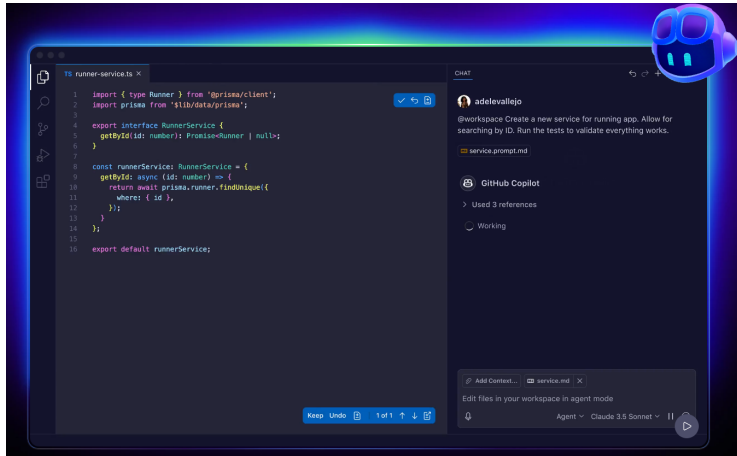
Instalación paso a paso
Casos de uso prácticos en R
Consideraciones
Ejercicios

¿Qué es GitHub Copilot?

¿Cómo funciona GitHub Copilot?

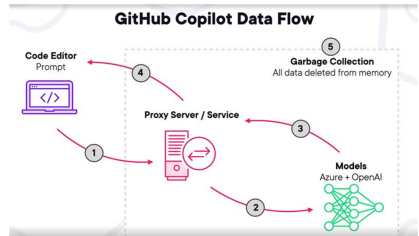
¿Por qué usar Copilot con R?

¿Qué es GitHub Copilot?



¿Cómo funciona GitHub Copilot?

- Utiliza un modelo de lenguaje entrenado con millones de líneas de código abierto.
- Predice y completa código basándose en el contexto local.
- Genera código a partir de comentarios en lenguaje natural.
- Aprende de patrones comunes en los lenguajes de programación.



¿Por qué usar Copilot con R?

- Facilita la escritura de código repetitivo o estructurado (e.g. funciones, bucles).
- Ayuda a descubrir funciones del ecosistema de R ('dplyr', 'ggplot2', etc.).
- Mejora la productividad y reduce errores comunes.
- Es útil como apoyo didáctico o para prototipado rápido en análisis de datos.

Instalación paso a paso

Instalación paso a paso

Requisitos previos

Paso 1: Activar GitHub Copilot
Paso 2: Activar GitHub Copilot en RStudio
Paso 3: Verificación rápida

Requisitos previos

- Tener una cuenta de GitHub.
- Suscripción activa a GitHub Copilot (gratuita para docentes y estudiantes).
- Tener instalado R y RStudio (la versión más reciente).
- Conexión a internet durante la programación.

Take flight with GitHub Copilot

For individuals For businesses

Free	Pro Most popular	Pro+
A fast way to get started with GitHub Copilot	Unlimited completions and chats with access to more models.	Maximum flexibility and model choice
\$0 ^{USD}	\$10 ^{USD} per month or \$100 per year	\$39 ^{USD} per month or \$390 per year
Get started	Try for 30 days free	Get started
Open in VS Code		
What's included ✓ 50 agent mode or chat requests per month ✓ 2,000 completions per month ✓ Access to Claude 3.5, Sonnet, GPT-4.1, and more	Everything in Free and: ✓ Unlimited agent mode and chats with GPT-4.1 ✓ Unlimited code completions ✓ Access to code review, Claude 3.7/4.0 Sonnet, Gemini 2.0 Pro, and more ✓ No more premium requests than Copilot Free to use the latest models, with the option to buy more Free for verified students, teachers, and maintainers of popular open source projects. Learn more	Everything in Pro and: ✓ Access to all models, including Claude 4.0 Opus, v3, and GPT-4.5 ✓ 30x more premium requests than Copilot Free to use the latest models, with the option to buy more ✓ Coding agent (preview)

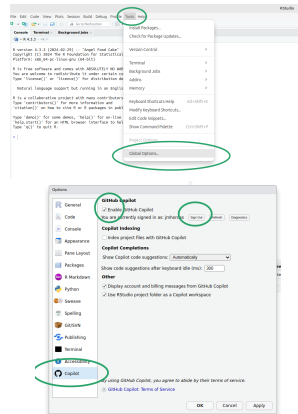
Paso 1: Activar GitHub Copilot

- Inicia sesión en tu cuenta de GitHub.
- Accede a Settings > Copilot.
- Elige la opción de suscripción (gratuita si eres docente o estudiante).
- Activa Copilot para tu cuenta personal o para organizaciones.

Puedes consultar: <https://github.com/features/copilot>

Paso 2: Activar GitHub Copilot en RStudio

- Abre RStudio.
- Tools → Global Options... → Copilot
- Clic en “Enable GitHub Copilot”
- Autoriza el inicio de sesión con GitHub.

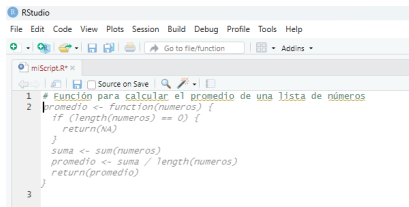


Paso 3: Verificación rápida

- Crea un archivo `script.R` nuevo.
- Escribe un comentario como:

Función para calcular el promedio de una lista de números

- Copilot debería sugerir automáticamente una función en R.
- Si no lo hace, revisa la configuración y permisos.



```
1 # Función para calcular el promedio de una lista de números
2 promedio <- function(numeros) {
3   if (length(numeros) == 0) {
4     return(NA)
5   }
6   suma <- sum(numeros)
7   promedio <- suma / length(numeros)
8   return(promedio)
9 }
```

Sugerencia de Copilot (*Ghost text*).

Casos de uso prácticos en R

Casos de uso prácticos en R

¿Qué puede hacer Copilot por ti en R?

- Síntesis de código
- Explicación/Documentación de código
- Completar bloques de código
- Identificación de errores
- Optimización de código
- Revisión de código
- Generación de tests
- Preguntas y respuestas con Copilot

Vamos a ver ejemplos prácticos.

Ejemplo 1: Síntesis de código

Objetivo: Generar código a partir de lenguaje natural.

- Escribimos:

```
# Read a CSV file
```

- Copilot sugiere:

```
data <- read.csv("data.csv")
```

- Escribimos:

```
# Show summary statistics
```

- Copilot sugiere:

```
summary(data)
```

Ejemplo 2: Síntesis de código

Objetivo: Generar código a partir de lenguaje natural.

Calcular la media ponderada de un vector de valores
y pesos

Copilot sugiere:

```
weighted_mean <- function(x, w) {  
  sum(x * w) / sum(w)  
}
```

Ejemplo 2: Síntesis de código

Cuidado! Comportamiento no determinista:

Calcular la media ponderada de un vector de valores
y pesos

```
weighted_mean <- function(values, weights) {  
  if (length(values) != length(weights)) {  
    stop("Values and weights must have the same length")  
  }  
  return(sum(values * weights) / sum(weights))  
}
```


Ejemplo 3: Síntesis de código

Objetivo: Generar código a partir de lenguaje natural.

- Escribimos:

```
# Calcular el factorial de un número
```

- Copilot sugiere:

```
factorial <- function(n) {  
  if(n == 0) {  
    return(1)  
  } else {  
    return(n * factorial(n-1))  
  }  
}
```

Ejemplo 3: Síntesis de código

Cuidado! Comportamiento no determinista

- Escribimos:

Calcular el factorial de un número

- Copilot sugiere:

```
factorial <- function(n) {  
  if (n < 0) {  
    stop("El número debe ser no negativo")  
  } else if (n == 0 || n == 1) {  
    return(1)  
  } else {  
    result <- 1  
    for (i in 2:n) {  
      result <- result * i  
    }  
    return(result)  
  }  
}
```

Ejemplo 4: Explicación/Documentación código

Objetivo: Entender qué hace el código.

- Tenemos:

```
# A script to tidy data
```

```
library(tidyverse)
```

```
data <- read.csv("data.csv")
```

```
filtered_data <- data %>%  
  filter(x > 10) %>%  
  arrange(desc(x))
```

- Colocando el cursor encima de `library(tidyverse)`, sugiere:

```
# Load the tidyverse package
```

Ejemplo 4: Explicación/Documentación código

Objetivo: Entender qué hace el código.

- Tenemos:

```
# A script to tidy data
```

```
library(tidyverse)
```

```
data <- read.csv("data.csv")
```

```
filtered_data <- data %>%  
  filter(x > 10) %>%  
  arrange(desc(x))
```

- Colocando el cursor encima de la siguiente línea, Copilot sugiere:

```
# Read the data from a CSV file
```

Ejemplo 4: Explicación/Documentación código

Objetivo: Entender qué hace el código.

- Tenemos:

```
# A script to tidy data
```

```
library(tidyverse)
```

```
data <- read.csv("data.csv")
```

```
filtered_data <- data %>%  
  filter(x > 10) %>%  
  arrange(desc(x))
```

- Colocando el cursor encima de la siguiente línea, Copilot sugiere:

```
# Filter rows where variable 'x' is greater than  
10 and arrange them in descending order
```

Ejemplo 5: Documentación de código

Objetivo: Documentar código

- Tenemos:

```
average <- function(x) {  
  n <- length(x)  
  if(n == 0) {  
    return(NA)  
  } else {  
    return(sum(x) / n)  
  }  
}
```

- Le pedimos:

```
# Add a roxygen-type documentation
```

Ejemplo 5: Documentación de código

Objetivo: Documentar código

- Copilot sugiere:

```
#' Calculate the average of a vector
#'  
#' @param x A numeric vector  
#' @return The average of the vector  
average <- function(x) {  
  n <- length(x)  
  if(n == 0) {  
    return(NA)  
  } else {  
    return(sum(x) / n)  
  }  
}
```

Ejemplo 6: Completar bloques de código

Objetivo: Completar bloques de código existente.

- Comenzamos escribiendo:

```
# Plot histogram for variable 'x'  
hist(  

```

- Copilot sugiere completarlo con:

```
hist(data$x, main="Histogram of x", xlab="x", col="blue")
```


Ejemplo 7: Identificar y corregir errores

Objetivo: Arreglar código existente.

- Tenemos:

```
add <- function(a, b) {  
  return(a + b)  
}  
print(add(5, "5"))
```

- Colocando el cursor debajo, Copilot responde:

```
# This will cause an error because "5" is a string,  
not a number.  
# To fix this, we can convert the string to a number  
using as.numeric()  
print(add(5, as.numeric("5")))
```

Ejemplo 8: Optimización de código

Objetivo: Mejorar la eficiencia del código

- Tenemos:

```
factorial <- function(n) {  
  if(n == 0) {  
    return(1)  
  } else {  
    return(n * factorial(n-1))  
  }  
}
```

- Escribimos:

```
# Optimiza la función
```

Ejemplo 8: Optimización de código

Objetivo: Mejorar la eficiencia del código

- Copilot sugiere:

```
factorial <- function(n) {  
  result <- 1  
  for(i in 1:n) {  
    result <- result * i  
  }  
  return(result)  
}
```

Ejemplo 9: Revisión de código

Objetivo: Corregir/Mejorar código existente

- Tenemos:

```
average <- function(x) {  
  return(sum(x) / length(x))  
}
```

- Le pedimos:

Revisa y mejora la función

- Copilot sugiere:

```
average <- function(x) {  
  n <- length(x)  
  if(n == 0) {  
    return(NA)  
  } else {  
    return(sum(x) / n)  
  }  
}
```

Ejemplo 10: Generación de tests

Objetivo: Generar casos de prueba

- Tenemos:

```
is_prime <- function(n) {  
  if (n <= 1) {  
    return(FALSE)  
  }  
  for (i in 2:sqrt(n)) {  
    if (n %% i == 0) {  
      return(FALSE)  
    }  
  }  
  return(TRUE)  
}
```

- Le pedimos:

```
# Generate some test cases
```

Ejemplo 10: Generación de tests

Objetivo: Generar casos de prueba

- Copilot sugiere:

```
# Test case 1: Check a prime number
print(is_prime(7)) # Expected output: TRUE
# Test case 2: Check a non-prime number
print(is_prime(4)) # Expected output: FALSE
# Test case 3: Check a negative number
print(is_prime(-7)) # Expected output: FALSE
# Test case 4: Check zero
print(is_prime(0)) # Expected output: FALSE
# Test case 5: Check one
print(is_prime(1)) # Expected output: FALSE
```

Preguntas y respuestas con Copilot

Uso de comentarios especiales:

```
data <- read.csv("data.csv")  
# Q:Cuál es la dimensión del dataframe?
```

Copilot responde:

A: La dimensión del dataframe es el número de filas y columnas

Q: ¿Cuántas filas y columnas tiene el dataframe?

A: El dataframe tiene 10 filas y 2 columnas

Consideraciones

Consideraciones al usar Copilot

¿Cómo interpreta Copilot el contexto?

- Usa el historial del archivo para entender qué tipo de código estás escribiendo.
- Interpreta comentarios, nombres de variables y funciones.
- Aprende del estilo y estructura de tu código.
- No siempre acierta, pero es útil como punto de partida.

Consejo: Corrige, modifica y verifica siempre lo que genera.

¿Cómo interpreta Copilot el contexto?

GitHub Copilot adapta sus sugerencias basándose en múltiples niveles de contexto:

- **Código alrededor del cursor:** Analiza el código antes y después del punto de edición.
- **Nombre y tipo de archivo:** Ajusta las sugerencias según el lenguaje y tipo de archivo (.R, .py, .html, etc.).
- **Pestañas abiertas:** Tiene en cuenta otros archivos abiertos para mantener la coherencia con la lógica del proyecto.
- **Estructura del proyecto y rutas de archivos:** Mejora sugerencias relacionadas con rutas de importación y relaciones entre módulos.
- **Frameworks y lenguajes detectados:** Ajusta el estilo y patrones del código según las tecnologías usadas (por ejemplo: tidyverse, Shiny, Flask, etc.).

¿Cómo interpreta Copilot el contexto?

¿Qué es una ventana de contexto?

- Es la cantidad de texto o código que el modelo puede “ver” y procesar al mismo tiempo.
- Afecta la capacidad del modelo para entender el entorno del código actual.

Tamaño típico de la ventana:

~200–500 líneas de código o ~4.000 tokens

¿Por qué es importante?

- Si hay demasiado código, el modelo puede perder información clave.
- Las sugerencias pueden volverse menos relevantes o imprecisas.

Buenas prácticas:

- Divide los problemas en prompts más pequeños.
- Sé explícito y enfocado en tus preguntas.

Alucinaciones en Copilot

Los modelos de lenguaje como GitHub Copilot pueden generar código que es **sintácticamente correcto pero semánticamente erróneo o sin sentido**, incluso si la instrucción es precisa.

¿Qué tipo de errores pueden producirse?

- **Funciones inexistentes:** sugerencias de funciones o paquetes que no existen.
- **Lógica incorrecta:** implementación errónea de algoritmos conocidos.

Alucinaciones en Copilot

Ejemplo de alucinación: Intentar usar una función que no existe

```
data <- read.csv("datos.csv")  
summary <- data_summary(data) # No existe en R
```

Corrección esperada:

```
summary <- summary(data)
```

Revisa siempre el código sugerido y verifica que las funciones realmente existen.

Buenas prácticas al usar Copilot

- **Revisa las sugerencias de código:** comprueba siempre que se ajustan a tus necesidades.
- **Personaliza mediante comentarios:** comentarios claros (# calcular media) guían a Copilot.
- **Considera la seguridad:** evita aceptar ciegamente comandos de exportación o conexiones externas.
- **Respetar derechos de autor y licencias:** no uses fragmentos idénticos sin asegurarte de tener permiso.
- **Aprovecha la oportunidad de aprendizaje:** analiza el código sugerido para mejorar tu dominio de R.

Ejercicios

Ejercicios usando Copilot

Ejercicio 1: Manipulación y visualización de datos

Usando Copilot (sin escribir ninguna línea de código): A partir del fichero `precio_bigmac_por_pais.csv`, crear un gráfico de barras vertical con los precios medios en euros por continente.

Requisitos del gráfico:

- Precio en euros.
- 5 continentes.
- Barras verticales ordenadas de mayor a menor.
- El precio medio exacto debe aparecer encima de cada barra.
- Barras de distintos colores (rojo y verde no pueden aparecer juntos).
- Guardar el gráfico en un fichero `.pdf`.

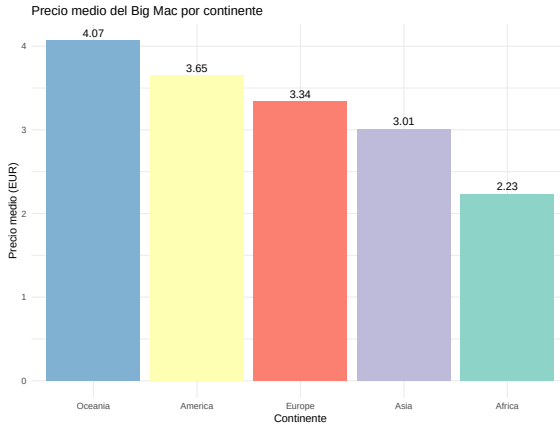
¿En qué continente se vende más cara la Big Mac? ¿Y dónde la más barata? ¿Cuánto cuesta en Europa?

Ejercicio 1: Manipulación y visualización de datos

Consejo: Divide y vencerás

- 1 Cargar el fichero de datos: `precio_bigmac_por_pais.csv`
- 2 Crear una nueva columna con los precios en euros.
- 3 Dividir el dataset por continentes.
- 4 Combinar los datos en un nuevo dataframe
- 5 Mostrar un gráfico de barras con los precios medios en euro por continente.
- 6 Adaptar el gráfico a los requisitos.

Ejercicio 1: Manipulación y visualización de datos



Ejercicio 1: Manipulación y visualización de datos

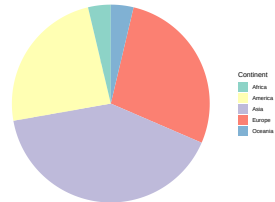
Realizar las siguientes consultas usando Copilot:

- 1 Comprobar que el número total de datos coincide con el número total de países del fichero .csv.
- 2 ¿Cuántos datos tiene cada continente?
- 3 ¿Cuál es el precio medio del Big Mac en Europa?

Ejercicio 1: Manipulación y visualización de datos

Objetivo: Generar un gráfico de tarta con el porcentaje de países por continente para mostrar la distribución de los datos.

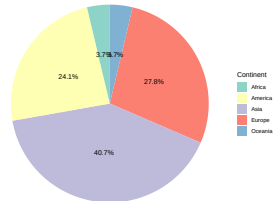
Porcentaje de países por continente



Ejercicio 1: Manipulación y visualización de datos

Objetivo: Modificar el gráfico de tarta para que muestre los porcentajes de cada porción.

Porcentaje de países por continente



Ejercicio 1: Manipulación y visualización de datos

Objetivo: Sustituir el gráfico de tarta por un gráfico de barras para mostrar los datos exactos del número de países por continente.

